SVKM's NMIMS

Mukesh Patel School of Technology, Management and Engineering

Department of Computer Engineering

CSEDS, 311, Sem V

Coursera Course: Database and SQL for Data Science with Python

Case Study Assignment (30 Marks)

Oct 7, 2026 ·

General Instructions

Each student designs, builds and queries a relational database for one case study, using Python to connect to SQL. Your case study depends on your roll number.

Roll number	Case study	Domain
Odd (1, 3, 5, ...)	Case Study A	City Care Multi-Specialty Hospital
Even (2, 4, 6, ...)	Case Study B	QuickBite Online Food Delivery

Tasks to complete

1. Read your case study and identify the entities, attributes and relationships.
2. Connect to a SQL database from Python. You may use sqlite3 (built in), mysql-connector-python or pymysql.
3. Create the 8 tables given in your case study, with suitable data types, PRIMARY KEY, FOREIGN KEY, NOT NULL and other constraints.
4. Insert at least 5 meaningful rows into each table using Python.
5. Execute the 5 queries given in your case study from Python and display their output.
6. Submit your work in the format given in the Submission Format section.

Rules

- Table and column names must be exactly as given. You may add extra columns, but do not remove any.
- Insert data in parent tables before child tables so foreign keys remain valid.

- Your data must be designed so that every query returns at least one row. Adjust your sample data if a query returns nothing.
- Every student must use their own sample data. Identical data or outputs across submissions will be treated as copying.

Learning outcomes: after this assignment, students will be able to translate a business scenario into a relational schema, implement it through Python-SQL connectivity, and write analytical queries using joins, aggregation, GROUP BY and HAVING.

Case Study A: City Care Multi-Speciality Hospital (Odd Roll Numbers)

Scenario. City Care is a 200-bed hospital that still tracks patients, doctors and billing in spreadsheets. Management wants a database to answer questions such as which departments are busiest, which doctors generate the most revenue, which medicines are prescribed most, and how long patients stay in each type of room.

The hospital has several departments, each on a floor and headed by a senior doctor. Every doctor belongs to one department and charges a fixed consultation fee. Patients register once and can book many appointments with different doctors; each appointment is Scheduled, Completed or Cancelled. During an appointment a doctor may prescribe several medicines with a quantity and dosage. Some patients are admitted to a room (General, Semi-Private, Private or ICU); each room belongs to a department and has a daily charge.

Tables to create

No.	Table name	Columns	Keys and constraints
1	Departments	dept_id, dept_name, floor_no, head_doctor_name	PK: dept_id; dept_name UNIQUE, NOT NULL
2	Doctors	doctor_id, doctor_name, dept_id, specialization, experience_years, consultation_fee	PK: doctor_id; FK: dept_id → Departments
3	Patients	patient_id, patient_name, gender, date_of_birth, city, phone	PK: patient_id; gender CHECK IN ('M','F','O')
4	Appointments	appointment_id, patient_id, doctor_id,	PK: appointment_id; FK: patient_id →

No.	Table name	Columns	Keys and constraints
		appointment_date, status	Patients, doctor_id → Doctors
5	Rooms	room_id, room_type, daily_charge, dept_id	PK: room_id; FK: dept_id → Departments
6	Admissions	admission_id, patient_id, room_id, admit_date, days_stayed	PK: admission_id; FK: patient_id → Patients, room_id → Rooms
7	Medicines	medicine_id, medicine_name, category, unit_price	PK: medicine_id; unit_price > 0
8	Prescriptions	prescription_id, appointment_id, medicine_id, quantity, dosage	PK: prescription_id; FK: appointment_id → Appointments, medicine_id → Medicines

Tasks to be performed:

1. Insert at least 5 rows in each table. Insert in this order: Departments, Doctors, Patients, Appointments, Rooms, Admissions, Medicines, Prescriptions.
2. Queries to execute

   Write the SQL for each query yourself and run it from Python. Each query needs at most one join.

1. Show each department's name and the number of doctors working in it. Display only departments that have more than one doctor.
2. Show each specialization and the average consultation fee of doctors in that specialization.
3. Show each room type and the total number of days patients stayed in rooms of that type.
4. Show each medicine's name and the total quantity prescribed. Display only medicines whose total quantity prescribed is more than 10.
5. Show each city and the number of patients from that city. Display only cities with at least 2 patients.

Case Study B: QuickBite Online Food Delivery (Even Roll Numbers)

Scenario. QuickBite is a food delivery start-up operating in three cities. Its data team needs a database to find top-earning restaurants, popular cuisines, the best delivery agents, preferred payment modes and high-value customers.

Customers sign up with their contact details and city. Partner restaurants list menu items, each with a category (Veg or Non-Veg) and a price. A customer places an order at one restaurant; an order can contain several menu items in different quantities. Each order is assigned to one delivery agent and has a status (Placed, Delivered or Cancelled). Every order has one payment record (UPI, Card, Cash or Wallet) with a status of Success or Failed. After delivery, customers may review the order with a 1 to 5 star rating.

Tables to create

No.	Table name	Columns	Keys and constraints
1	Customers	customer_id, customer_name, email, phone, city, join_date	PK: customer_id; email UNIQUE
2	Restaurants	restaurant_id, restaurant_name, city, cuisine_type, avg_rating	PK: restaurant_id; avg_rating between 0 and 5
3	MenuItems	item_id, restaurant_id, item_name, category, price	PK: item_id; FK: restaurant_id → Restaurants; category CHECK IN ('Veg','Non-Veg')
4	DeliveryAgents	agent_id, agent_name, phone, vehicle_type, city	PK: agent_id
5	Orders	order_id, customer_id, restaurant_id, agent_id, order_date, order_status	PK: order_id; FK: customer_id → Customers, restaurant_id → Restaurants, agent_id → DeliveryAgents
6	OrderItems	order_item_id, order_id, item_id, quantity	PK: order_item_id; FK: order_id → Orders, item_id →

No.	Table name	Columns	Keys and constraints
			MenuItems; quantity > 0
7	Payments	payment_id, order_id, payment_mode, amount, payment_status	PK: payment_id; FK: order_id → Orders
8	Reviews	review_id, order_id, customer_id, rating, comments	PK: review_id; FK: order_id → Orders, customer_id → Customers; rating between 1 and 5

Tasks to be performed:

1. Insert at least 5 rows in each table. Insert in this order: Customers, Restaurants, MenuItems, DeliveryAgents, Orders, OrderItems, Payments, Reviews

2. Queries to execute

   Write the SQL for each query yourself and run it from Python. Each query needs at most one join.

1. Show each restaurant's name, the number of menu items it offers and the average price of those items.
2. Show each payment mode and the total amount collected through it. Display only payment modes whose total amount is more than ₹500.
3. Show each customer's name and the number of orders they placed. Display only customers who placed 2 or more orders.
4. Show each delivery agent's name and the number of orders assigned to them.
5. Show each menu category (Veg / Non-Veg), the number of items in it and the highest item price.

Submission Format

Submit one PDF report plus your code file (.ipynb or .py), arranged in the 7 parts below. Every code part must be followed by a screenshot or copied output proving it ran.

Part 1: Cover page

Field	Details
Student name	
Roll number	
Odd / Even	
Case study allotted	A (Hospital) or B (Food Delivery)
Database used	SQLite / MySQL
Date of submission	

Part 2: Schema design

List your 8 tables with each column's data type and constraints, and write 2 to 3 lines explaining the relationships between tables. An ER diagram (hand-drawn or tool-made) earns bonus credit.

Part 3: Python-SQL connectivity code

Paste your Python code that imports the database library and connects to the database, followed by the output confirming the connection.

Part 4: Table creation code

Paste your Python code that creates all 8 tables with their constraints, followed by output proving that all 8 tables exist in the database.

Part 5: Data insertion code

Paste your Python code that inserts at least 5 rows into each table, followed by the contents of every table displayed from Python.

Part 6: Query execution and results

For each of the 5 queries, paste (a) the question, (b) the SQL query you wrote, (c) the Python code that executes it and (d) the output. Then summarise your results in this table:

Query	Output (paste table or screenshot)	Rows returned	Your 1-line interpretation
Q1			
Q2			
Q3			

Query	Output (paste table or screenshot)	Rows returned	Your 1-line interpretation
Q4			
Q5			

Part 7: Conclusion

In 5 to 8 lines, state what you learned, any errors you faced (for example a foreign key violation or a query returning no rows) and how you fixed them.

Submission guidelines

- File names: RollNo_Name_CaseStudyA.pdf and RollNo_Name_CaseStudyA.ipynb (use B if allotted Case Study B).
- Attempting the wrong case study for your roll number earns zero for Parts 4 to 6.
- **submission deadline: 21/10/2026**